# Lab 6: SPI SRAM Module

ECEN 127L: Tuesday 5:15 - 8:00 pm

Ryan Kiniris and Dylan Thornburg

# Introduction

In this lab, we  are creating, implementing, and verifying a synthesizable Verilog model of the Microchip 23A640 SPI SRAM, in which is a 64-Kbit memory that communicates through a 4-wire Serial Peripheral Interface.

The goal is to produce a fully functional chip model that could communicate with an SPI main controller, as well as accurately follow the timing diagrams that are given to us in the datasheet, and pass the testbench that we have made. The project established a practical understanding of serial communication protocols, bidirectional data buses, and hardware-accurate state-machine design.

# Procedure

We had started out by designing the  8K × 8 asynchronous SRAM module(64-Kbit memory) that stored 8-bit data words using a single bidirectional data bus for both the read and write operations. After verifying the right operation with our own made testbench, we then made an SPI interface that uses the serial communication through both registers and state machines managing signals like CSB, SCK, SI, SO, and HOLD b. This interface moves the data in and out based on clock edges and supports both READ and WRITE commands. Finally, we then incorporated the SRAM and SPI modules into a top-level M23A640 design, added logic to convert SPI instructions into memory operations, and hard-wired the status register to 00000011. The combined design was tested using the given tbserial.v testbench to confirm proper timing, high-impedance states, and full compliance with the 23A640 datasheet.

# Results

- **What problems did you encounter in part 1 while testing your steps yourself?**
  For part 1, when it came to SRAM we came into an issue with the write enable, as with the write enable in our testbench we had after the set data and testbench which would delay our response by 1 set. We also didn't initially update our mem [address] at the "we" posedge and instead simply when "we" was asserted.

- **Did any problems arise when demonstrating for the TA for part 1? What were they? Explain your thoughts on how/why these test cases escaped your own testing.**

  When demoing with the TA, there were no real issues besides updating our architecture to match the datasheet more closely.

- **What problems did you encounter in parts 2/3 while testing your steps yourself?**

There were several debugging issues we encountered. At first we tried to implement rd_data and wr_data buses for the single bidirectional data before realizing it was much easier and schematically correct to make another data inout on the spi module.
We also had an error with the wr output. We accidentally forgot to include some logic to only write when data_count ==8. It kept writing the wrong data over and over. Once we fixed these things, our spi and top level module worked as intended.

- **Did any problems arise when demonstrating for the TA for parts 2/3? What were they? Explain your thoughts on how/why these test cases escaped your own testing.**

  No issues demoing here.

# SIMULATION RESULTS

## SRAM TB Results:

```
[dthornburg@linux402604 Lab6]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;  Nov  4 18:19 2025


SRAM testbench start
addr=0000, re=0, we=0, data=zz, @                   0


Write at 0000 = A5
addr=0000, re=0, we=1, data=a5, @                   5
addr=0000, re=0, we=0, data=zz, @                  10


Write at 0001 = 5A
addr=0001, re=0, we=1, data=5a, @                  15
addr=0001, re=0, we=0, data=zz, @                  20


Write at 1ABC = 3C
addr=1abc, re=0, we=1, data=3c, @                  25
addr=1abc, re=0, we=0, data=zz, @                  30


Write at 1FFF = FF
addr=1fff, re=0, we=1, data=ff, @                  35
addr=1fff, re=0, we=0, data=zz, @                  40


Read at 0000 expect A5
addr=0000, re=1, we=0, data=a5, @                  45
addr=0000, re=0, we=0, data=zz, @                  50


Read at 0001 expect 5A
addr=0001, re=1, we=0, data=5a, @                  55
addr=0001, re=0, we=0, data=zz, @                  60


Read at 1ABC expect 3C
addr=1abc, re=1, we=0, data=3c, @                  65
addr=1abc, re=0, we=0, data=zz, @                  70


Read at 1FFF expect FF
addr=1fff, re=1, we=0, data=ff, @                  75
addr=1fff, re=0, we=0, data=zz, @                  80


Both re and we high test driving from tb (expect ff)
addr=0000, re=1, we=1, data=ff, @                  85
addr=0000, re=0, we=0, data=zz, @                  90


Both re and we high test pulling from dut (expect zz)
addr=0000, re=1, we=1, data=zz, @                  95
addr=0000, re=0, we=0, data=zz, @                 100


SRAM testbench end
$finish called from file "tbsram.v", line 107.
$finish at simulation time                  105
           V C S   S i m u l a t i o n   R e p o r t
Time: 105
CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
Tue Nov  4 18:19:11 2025
[dthornburg@linux402604 Lab6]$
```

**Top Level Module M23A640 Results:**

```
[dthornburg@linux402604 Lab6]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;
  Nov 11 17:28 2025
Beginning write-read test
Passed write-read test
Beginning write-read test 2
Passed write-read test 2
Beginning broken write test
Passed broken write test
Beginning alias test
Passed alias test
***** - ALL TESTS PASSED - *****
$finish called from file "tbserial.v", line 27.
$finish at simulation time                20825
           V C S   S i m u l a t i o n   R e p o r t
Time: 20825
CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
Tue Nov 11 17:28:57 2025
[dthornburg@linux402604 Lab6]$
```

# Conclusion

Overall, in this lab we had designed, implemented, and verified a fully synthesizable Verilog model of the Microchip 23A640 64-Kbit SPI SRAM device. The project was divided into three major parts, creating an 8K×8 SRAM memory block, building the SPI serial communication interface, and then integrating both components into the top-level M23A640 module. Throughout the process, we ensured that all timing, signal transitions, and data exchanges matched the datasheet specifications, including proper high-impedance states and edge-triggered operations. Our design fully supported the required READ and WRITE instructions, while simplifying features like the status register to a fixed value of 00000011. We also double checked the correctness of the design through detailed testbenches that simulated both write and read transactions, making sure the SPI protocol and memory access behaved as expected. The final simulation passed the Wolfe's provided testbench, demonstrating accurate signal timing and reliable communication between the SPI interface and SRAM.

# Appendix

## SRAM CODE

```
module sram(
        input [12:0] address,
        input re,
        input we,
        inout [7:0] data  //bidirectional
);
        reg [7:0] mem [0:8191];
        assign data = (re && !we) ? mem[address] : 8'bz;
        always@(posedge we) if (!re) mem[address] = data;
endmodule
```

```
module sram_testbench();

reg [12:0] address;
reg re, we;
wire [7:0] data;
//tri state signals
reg [7:0] tb_data;
reg tb_driver; //1 = tb drives | 0 = sram dut module drives
assign data = tb_driver ? tb_data : 8'hzz;

sram dut(address, re, we, data);

initial begin

$display("\n");
$display("SRAM testbench start");
$monitor("addr=%h, re=%b, we=%b, data=%h, @%d", address, re, we, data, $time);

address = 13'd0; re = 1'b0; we = 1'b0; tb_data = 8'h00; tb_driver = 1'b0;
#5;
$display("\n");

//tb_data MUST come before we due to posedge writing. Consider non blocking for future tbs

// write 1
$display("Write at 0000 = A5");
address = 13'h0000; re = 1'b0; tb_data = 8'hA5; tb_driver = 1'b1; we = 1'b1;
#5;
we = 1'b0; tb_driver = 1'b0;
#5;
$display("\n");

// write 2
$display("Write at 0001 = 5A");
address = 13'h0001; re = 1'b0; tb_data = 8'h5A; tb_driver = 1'b1; we = 1'b1;
#5;
we = 1'b0; tb_driver = 1'b0;
#5;
```

```
$display("\n");

// write 3
$display("Write at 1ABC = 3C");
address = 13'h1ABC; re = 1'b0; tb_data = 8'h3C; tb_driver = 1'b1; we = 1'b1;
#5;
we = 1'b0; tb_driver = 1'b0;
#5;
$display("\n");

// write 4
$display("Write at 1FFF = FF");
address = 13'h1FFF; re = 1'b0; tb_data = 8'hFF; tb_driver = 1'b1; we = 1'b1;
#5;
we = 1'b0; tb_driver = 1'b0;
#5;
$display("\n");

// read 1
$display("Read at 0000 expect A5");
address = 13'h0000; re = 1'b1; we = 1'b0; tb_driver = 1'b0;
#5;
re = 1'b0;
#5;
$display("\n");

// read 2
$display("Read at 0001 expect 5A");
address = 13'h0001; re = 1'b1; we = 1'b0; tb_driver = 1'b0;
#5;
re = 1'b0;
#5;
$display("\n");

// read 3
$display("Read at 1ABC expect 3C");
address = 13'h1ABC; re = 1'b1; we = 1'b0; tb_driver = 1'b0;
#5;
re = 1'b0;
#5;
$display("\n");

// read 4
$display("Read at 1FFF expect FF");
address = 13'h1FFF; re = 1'b1; we = 1'b0; tb_driver = 1'b0;
#5;
re = 1'b0;
#5;
$display("\n");

// both re & we high
$display("Both re and we high test driving from tb (expect ff b/c of tb structure, but it really doesnt
write anything)");
address = 13'h0000; re = 1'b1; we = 1'b1; tb_driver = 1'b1;
#5;
```

```
re = 1'b0; we = 1'b0; tb_driver = 1'b0;
#5;
$display("\n");

// both re & we high
$display("Both re and we high test pulling from dut (expect zz)");
address = 13'h0000; re = 1'b1; we = 1'b1; tb_driver = 1'b0;
#5;
re = 1'b0; we = 1'b0;
#5;
$display("\n");


$display("SRAM testbench end");
$finish;
end

endmodule
```

## SPI CODE

```
module spi(
        input csb,
        input sck,
        input holdb,
        input si,
        inout [7:0] data,
        output reg so,
        output reg [15:0] address,
        output reg we,
        output reg re
);
        reg [7:0] instr;
        reg [3:0] instr_count;
        reg [4:0] addr_count;
        reg [3:0] data_count;

        reg [7:0] data_driver;
        assign data = data_driver;

        always@(negedge csb) begin
                instr <= 8'b0;
                address <= 16'b0;
                data_driver <= 8'b0;
                instr_count <= 4'b0;
                addr_count <= 5'b0;
```

```
		data_count <= 4'b0;
	end

	//input si on posedge
	always@(posedge sck) begin
		if (~csb && holdb) begin
			if (instr_count < 4'd8) begin
				instr_count <= instr_count + 1; //count till all 8 bits are passed for instr
				instr <= (instr << 1) | {7'b0, si};
			end
			if (instr_count == 4'd8 && addr_count < 5'd16) begin
				addr_count <= addr_count +1; //count till full adress received
				address <= (address << 1) | {15'b0, si};
			end
			if (instr_count == 4'd8 && addr_count == 5'd16 && data_count < 4'd8) begin
				if (instr == 8'b00000010) begin
					data_count <= data_count +1; //get data to write
					data_driver <= (data_driver << 1) | {7'b0, si};
				end
			end
		end
	end

	//output so on negedge. need blocking assignments because order is integral to my indexing
logic
	always@(negedge sck) begin
		if (~csb && holdb) begin
			if (instr_count == 4'd8 && addr_count == 5'd16 && data_count <= 4'd8) begin
				if (instr == 8'b00000011) begin
					data_count = data_count + 1;
					so = data[4'd8-data_count];
				end
			end
			else so <= 1'bz;
		end
	end


	always@(*) begin
		if (instr_count == 4'd8 && addr_count == 5'd16) begin
			if (instr == 8'b00000010 && data_count == 4'd8) begin //data only ready when
all 8 in driver
				we <=1;
				re <=0;
			end
			if (instr == 8'b00000011) begin //data ready immediately. count used for output
on so in negedge sck
				we <=0;
				re <=1;
				data_driver <= 8'hzz;
			end
		end
		else begin
			we <=0;
			re <=0;
```

```
                end
        end
endmodule
```

## M23A640 Code

```
module M23A640(csb, so, holdb, sck, si);
        input csb;
        output so;
        input holdb;
        input sck;
        input si;
        wire [15:0] address;
        wire we, re;
        wire [7:0] data;

        spi pi(
                .csb(csb),
                .sck(sck),
                .holdb(holdb),
                .si(si),
                .data(data),
                .so(so),
                .address(address),
                .we(we),
                .re(re)
        );

        sram gskill(
                .address(address[12:0]),
                .re(re),
                .we(we),
                .data(data)
        );

endmodule
```

```
module testbench();


reg csb, sck, si;
wire so;
reg [7:0] dreg;
reg fail;
```

```
reg anyfail;



M23A640 dut(csb, so, 1'b1, sck, si);

initial begin
  anyfail = 0;
  csb = 1;
  sck = 1; #5 sck = 0;
  write_read_test();
  write_read_test2();
  broken_write_test();
  alias_test();
  #5 sck = 1; #5 sck = 0;
  #5 sck = 1; #5 sck = 0;
  #5 sck = 1; #5 sck = 0;
  if (anyfail) $display("XXXXX - Failed to pass test suite - XXXXX");
  else $display("***** - ALL TESTS PASSED - *****");
$finish;
end


task write_read_test();
  begin
  // Perform 10 writes then 10 reads to see if the data is read back correctly
    $display("Beginning write-read test");
//    $monitor("dreg = %h, @ %d", dreg,$time);
    fail = 0;
    write(16'h0001, 8'h01);
    write(16'h0002, 8'h12);
    write(16'h0003, 8'h13);
    write(16'h0003, 8'h23);
    write(16'h0004, 8'h34);
    write(16'h0005, 8'h45);
    write(16'h1001, 8'h56);
    write(16'h1002, 8'h67);
    write(16'h1003, 8'h78);
    write(16'h1004, 8'h89);
    write(16'h1005, 8'h9A);
    read(16'h0001, dreg);  if (dreg !== 8'h01) fail = 1;
    read(16'h0002, dreg);  if (dreg !== 8'h12) fail = 1;
    read(16'h0003, dreg);  if (dreg !== 8'h23) fail = 1;
    read(16'h0004, dreg);  if (dreg !== 8'h34) fail = 1;
    read(16'h0005, dreg);  if (dreg !== 8'h45) fail = 1;
    read(16'h1001, dreg);  if (dreg !== 8'h56) fail = 1;
    read(16'h1002, dreg);  if (dreg !== 8'h67) fail = 1;
    read(16'h1003, dreg);  if (dreg !== 8'h78) fail = 1;
    read(16'h1004, dreg);  if (dreg !== 8'h89) fail = 1;
    read(16'h1005, dreg);  if (dreg !== 8'h9A) fail = 1;
    if (fail == 1) begin
      anyfail = 1;
      $display("XXXXX - Failed write-read test - XXXXX");
    end
    else $display("Passed write-read test");
```

```
  end
endtask

task write_read_test2();
  begin
  // Perform some re-writes to if the data is read back correctly
    $display("Beginning write-read test 2");
//    $monitor("dreg = %h, @ %d", dreg,$time);
    fail = 0;
    write(16'h0001, 8'h01);
    write(16'h0002, 8'h12);
    write(16'h0003, 8'h23);
    write(16'h0004, 8'h34);
    write(16'h0005, 8'h45);
    read(16'h0001, dreg);  if (dreg !== 8'h01) fail = 1;
    read(16'h0002, dreg);  if (dreg !== 8'h12) fail = 1;
    read(16'h0003, dreg);  if (dreg !== 8'h23) fail = 1;
    read(16'h0004, dreg);  if (dreg !== 8'h34) fail = 1;
    read(16'h0005, dreg);  if (dreg !== 8'h45) fail = 1;
    write(16'h0001, 8'h10);
    write(16'h0002, 8'h21);
    write(16'h0003, 8'h32);
    read(16'h0001, dreg);  if (dreg !== 8'h10) fail = 1;
    read(16'h0002, dreg);  if (dreg !== 8'h21) fail = 1;
    read(16'h0003, dreg);  if (dreg !== 8'h32) fail = 1;
    read(16'h0004, dreg);  if (dreg !== 8'h34) fail = 1;
    read(16'h0005, dreg);  if (dreg !== 8'h45) fail = 1;
    if (fail == 1) begin
      anyfail = 1;
      $display("XXXXX - Failed write-read test 2 - XXXXX");
    end
    else $display("Passed write-read test 2");
  end
endtask

task broken_write_test();
  begin
// Test response to a broken write
    $display("Beginning broken write test");
    fail = 0;
    write(16'h0001, 8'h01);
    write(16'h0002, 8'h12);
    read(16'h0001, dreg);  if (dreg !== 8'h01) fail = 1;
    read(16'h0002, dreg);  if (dreg !== 8'h12) fail = 1;
    write(16'h0001, 8'h34);
    broken_write(16'h0002, 8'h56);
    write(16'h0003, 8'h78);
    read(16'h0001, dreg);  if (dreg !== 8'h34) fail = 1;
    read(16'h0002, dreg);  if (dreg !== 8'h12) fail = 1;
    read(16'h0003, dreg);  if (dreg !== 8'h78) fail = 1;
    if (fail == 1) begin
      anyfail = 1;
      $display("XXXXX - Failed broken write test - XXXXX");
    end
    else $display("Passed broken write test");
```

```
  end
endtask

task alias_test();
  begin
// Test alias behavior
    $display("Beginning alias test");
    fail = 0;
    write(16'h0001, 8'h01);
    write(16'h0002, 8'h12);
    read(16'h0001, dreg);  if (dreg !== 8'h01) fail = 1;
    read(16'h0002, dreg);  if (dreg !== 8'h12) fail = 1;
    read(16'h8001, dreg);  if (dreg !== 8'h01) fail = 1;
    read(16'h8002, dreg);  if (dreg !== 8'h12) fail = 1;
    read(16'h4002, dreg);  if (dreg !== 8'h12) fail = 1;
    write(16'h4001, 8'h34);
    write(16'h4002, 8'h56);
    read(16'h0001, dreg);  if (dreg !== 8'h34) fail = 1;
    read(16'h0002, dreg);  if (dreg !== 8'h56) fail = 1;
    read(16'h8001, dreg);  if (dreg !== 8'h34) fail = 1;
    read(16'h8002, dreg);  if (dreg !== 8'h56) fail = 1;
    read(16'h4002, dreg);  if (dreg !== 8'h56) fail = 1;
     if (fail == 1) begin
      anyfail = 1;
      $display("XXXXX - Failed alias test - XXXXX");
    end
    else $display("Passed alias test");
  end
endtask

task write(input [15:0] address, input [7:0] data);
  begin
    csb = 0;                                    // Start Transaction
    si = 0; #5 sck = 1;                 // Send write instruction 0000 0010
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 1; sck = 0; #5 sck = 1;
    #5 si = 0; sck = 0; #5 sck = 1;

    #5 si = address[15]; sck = 0; #5 sck = 1;       // Send address MSB first
    #5 si = address[14]; sck = 0; #5 sck = 1;
    #5 si = address[13]; sck = 0; #5 sck = 1;
    #5 si = address[12]; sck = 0; #5 sck = 1;
    #5 si = address[11]; sck = 0; #5 sck = 1;
    #5 si = address[10]; sck = 0; #5 sck = 1;
    #5 si = address[9]; sck = 0; #5 sck = 1;
    #5 si = address[8]; sck = 0; #5 sck = 1;
    #5 si = address[7]; sck = 0; #5 sck = 1;
    #5 si = address[6]; sck = 0; #5 sck = 1;
    #5 si = address[5]; sck = 0; #5 sck = 1;
    #5 si = address[4]; sck = 0; #5 sck = 1;
    #5 si = address[3]; sck = 0; #5 sck = 1;
```

```
    #5 si = address[2]; sck = 0; #5 sck = 1;
    #5 si = address[1]; sck = 0; #5 sck = 1;
    #5 si = address[0]; sck = 0; #5 sck = 1;


    #5 si = data[7]; sck = 0; #5 sck = 1;  // Send data MSB first
    #5 si = data[6]; sck = 0; #5 sck = 1;
    #5 si = data[5]; sck = 0; #5 sck = 1;
    #5 si = data[4]; sck = 0; #5 sck = 1;
    #5 si = data[3]; sck = 0; #5 sck = 1;
    #5 si = data[2]; sck = 0; #5 sck = 1;
    #5 si = data[1]; sck = 0; #5 sck = 1;
    #5 si = data[0]; sck = 0; #5 sck = 1;
    #5 sck = 0; #5 csb = 1; #5;
  end
endtask


task read(input [15:0] address, output reg [7:0] data);
  begin
    csb = 0;                                    // Start Transaction
    si = 0; #5 sck = 1;                 // Send read instruction 0000 0011
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 1; sck = 0; #5 sck = 1;
    #5 si = 1; sck = 0; #5 sck = 1;

    #5 si = address[15]; sck = 0; #5 sck = 1;       // Send address MSB first
    #5 si = address[14]; sck = 0; #5 sck = 1;
    #5 si = address[13]; sck = 0; #5 sck = 1;
    #5 si = address[12]; sck = 0; #5 sck = 1;
    #5 si = address[11]; sck = 0; #5 sck = 1;
    #5 si = address[10]; sck = 0; #5 sck = 1;
    #5 si = address[9]; sck = 0; #5 sck = 1;
    #5 si = address[8]; sck = 0; #5 sck = 1;
    #5 si = address[7]; sck = 0; #5 sck = 1;
    #5 si = address[6]; sck = 0; #5 sck = 1;
    #5 si = address[5]; sck = 0; #5 sck = 1;
    #5 si = address[4]; sck = 0; #5 sck = 1;
    #5 si = address[3]; sck = 0; #5 sck = 1;
    #5 si = address[2]; sck = 0; #5 sck = 1;
    #5 si = address[1]; sck = 0; #5 sck = 1;
    #5 si = address[0]; sck = 0; #5 sck = 1;


    #5 sck = 0; #5 data[7] = so; sck = 1; // Receive data MSB first
    #5 sck = 0; #5 data[6] = so; sck = 1;
    #5 sck = 0; #5 data[5] = so; sck = 1;
    #5 sck = 0; #5 data[4] = so; sck = 1;
    #5 sck = 0; #5 data[3] = so; sck = 1;
    #5 sck = 0; #5 data[2] = so; sck = 1;
    #5 sck = 0; #5 data[1] = so; sck = 1;
```

```
    #5 sck = 0; #5 data[0] = so; sck = 1;
    #5 sck = 0; #5 csb = 1; #5;
//  $display("performing read from %h, data is %h",address, data);
  end
endtask

task broken_write(input [15:0] address, input [7:0] data);
  begin
    csb = 0;                                    // Start Transaction
    si = 0; #5 sck = 1;                 // Send write instruction 0000 0010
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 0; sck = 0; #5 sck = 1;
    #5 si = 1; sck = 0; #5 sck = 1;
    #5 si = 0; sck = 0; #5 sck = 1;

    #5 si = address[15]; sck = 0; #5 sck = 1;       // Send address MSB first
    #5 si = address[14]; sck = 0; #5 sck = 1;
    #5 si = address[13]; sck = 0; #5 sck = 1;
    #5 si = address[12]; sck = 0; #5 sck = 1;
    #5 si = address[11]; sck = 0; #5 sck = 1;
    #5 si = address[10]; sck = 0; #5 sck = 1;
    #5 si = address[9]; sck = 0; #5 sck = 1;
    #5 si = address[8]; sck = 0; #5 sck = 1;
    #5 si = address[7]; sck = 0; #5 sck = 1;
    #5 si = address[6]; sck = 0; #5 sck = 1;
    #5 si = address[5]; sck = 0; #5 sck = 1;
    #5 si = address[4]; sck = 0; #5 sck = 1;
    #5 si = address[3]; sck = 0; #5 sck = 1;
    #5 si = address[2]; sck = 0; #5 sck = 1;
    #5 si = address[1]; sck = 0; #5 sck = 1;
    #5 si = address[0]; sck = 0; #5 sck = 1;


    #5 si = data[7]; sck = 0; #5 sck = 1;  // Send data MSB first
    #5 si = data[6]; sck = 0; #5 sck = 1;
    #5 si = data[5]; sck = 0; #5 sck = 1;
    #5 si = data[4]; sck = 0; #5 sck = 1;
    #5 si = data[3]; sck = 0; #5 sck = 1;
    #5 si = data[2]; sck = 0; #5 sck = 1;
    #5 si = data[1]; sck = 0; #5 sck = 1;csb = 1;
    #5 si = data[0]; sck = 0; #5 sck = 1;
    #5 sck = 0; #5 ; #5;
  end
endtask
endmodule
```